University of Camerino

School of Science and Technologies

Course Master Degree In Computer Science (LM-18)
Technologies for Big Data Management

IOT#ETL Thingsboard

Student
Alessio Re

Registration 135435

Supervisor
Massimo Callisto De Donato

A.Y. 2025/2026

Abstract

This project, titled **IoT#ETL with Thingsboard** aims to design and implement a bidirectional synchronization system between an IoT platform, ThingsBoard, and a graph database. The primary objective is to solve the limitation of standard relational models in handling complex IoT infrastructures by applying graph theory for data persistence. The system utilizes an ETL (Extract, Transform, Load) pipeline to pull asset and device configurations from ThingsBoard via REST APIs and model them as nodes and relationships in a graph database. Furthermore, a custom User Interface (UI) is developed to visualize, modify, and manage these relationships dynamically, with a feedback loop that updates the source IoT platform. This architecture enables advanced topology management for complex environments, such as the University of Camerino's campus.

Introduction

The management of Big Data in IoT environments often struggles with the rigidity of standard database schemas when representing deep, nested hierarchies. This project utilizes ThingsBoard, an open-source IoT platform, to manage device telemetry and state. However, to enable complex relationship analysis and flexible topology management, the project introduces a dedicated persistence layer based on a graph database.

The core functionality revolves around a "Digital Twin" concept for the Polo Lodovici infrastructure. The system models physical **assets** (e.g., "First Floor", "AB1 Classroom") and their containment **relationships** with IoT devices (e.g., Temperature Sensors, Gateways).

The key architectural components are defined in the following modules (Figure 1):

Source System Acts as the authoritative source for device inventory and telemetry. It exposes data via a secure REST API (through Thingsboard).

ETL Middleware A Python-based module responsible for authenticating with ThingsBoard, extracting the JSON-based entity list, transforming the logical links into graph edges, and loading them into the database.

Graph Database Stores the infrastructure as a property graph, allowing for high-performance traversal of the campus hierarchy.

Management UI A web-based interface allowing operators to perform CRUD operations on the graph, such as moving a sensor from one room to another.

Reverse Sync Module Ensures data consistency by exporting structural changes made in the UI back to ThingsBoard.

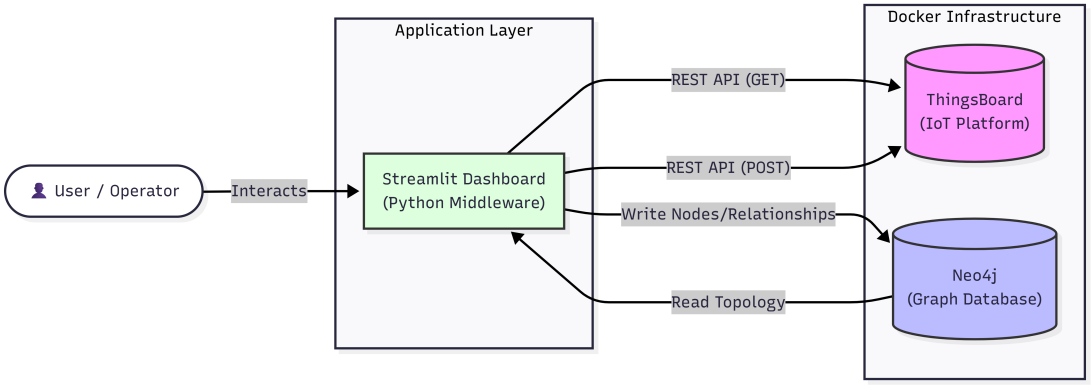

Figure 1: Project Architecture.

Roadmap

The implementation is divided into four distinct phases, ensuring a logical progression from infrastructure setup to full bidirectional synchronization.

1. **Environment Setup and Data Modeling** - Deployment of containerized services (Docker) for ThingsBoard and the graph database. Configuration of the initial source data in ThingsBoard to match the "Polo Lodovici" specification (Assets: Floors, Rooms; Devices: Sensors, Gateways). Definition of the graph schema.

2. **One-Way ETL Implementation (ThingsBoard → Graph)** - Development of the Python connector to interact with ThingsBoard REST APIs. Implementation of the "Extract" logic to fetch all tenants, assets, and devices. Implementation of the "Transform" logic to resolve ID-based references into graph relationships. Implementation of the "Load" logic to persist the topology into the graph database.

3. **User Interface & Graph Manipulation** - Development of a dashboard (UI) to visualize the current graph topology. Implementation of interactive features allowing users to add new nodes (e.g., new rooms) or modify existing relationships (e.g., moving a device). Local persistence of these changes within the graph database.

4. **Reverse Synchronization (Graph → ThingsBoard)** - Development of the "Export" module to detect changes in the graph database. Integration of ThingsBoard POST APIs to push updates back to the source platform, ensuring the digital twin remains synchronized with the management layer

Project Setup

After all the introduction, I've proceeded with the setup of the project code. I've installed Docker and succesfully set up Thingsboard in my application. Then I've installed Neo4j in the application to use it as the persistence layer. After ThingsBoard was populated with Assets, Devices, and Relations given by the specification, I will develop a basic **Export** module to test Thingsboard REST API.

Now that I can fully use ThingsBoard, I can move to the core of the assignment: The ETL Process. The development goal now is to take the flat list of assets I've just insterted and reconstruct the graph inside Neo4j with all the needed features.

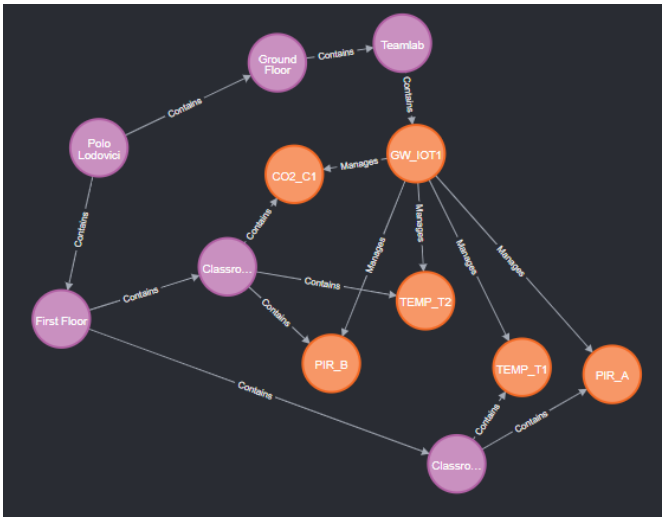

Figure 2: Graph obtained by adding all the assets and relations to Neo4j

Frontend Integration

The first prototype's UI consists of 2 components (Figure 3):

1. A **sidebar** (made for configuration and to run synchronization);

2. The **main interface**.

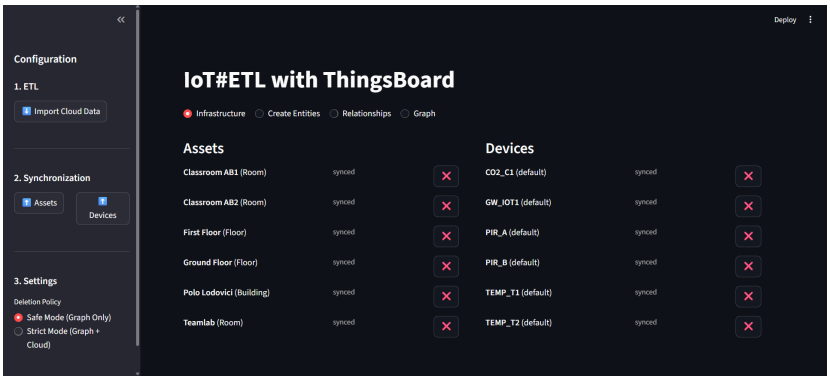

Figure 3: Implemented User Interface (v1.0)

As in the picture, the first component is split into:

- **Import Cloud Data button (ETL)**: this button pulls an existing infrastructure from ThingsBoard. It fetches Assets, Devices, and Relationships to populate the local Neo4j graph;

- **Synchronization**: buttons here upload all locally created draft entities to the cloud;

- **Deletion Policy section**: this section is specifically made to choose between "Safe Mode" or "Strict Mode" deletion policy. The first one deletes nodes or relationships from the local graph, while the second option deletes the chosen entity from Thingsboard (so it's considered critical).

The main interface comprehend 4 different tabs.

1. **Infrastructure** tab (Figure 3): this is specifically made to show Assets and Devices from Thingsboard, and it can be used to check whether the elements in it are synced to the cloud or just draft items. Furthermore it is used to delete elements from the graph (depending on deletion policy)

2. **"Create Entities"** tab: its purpose is to create draft items, such as Assets or Devices. Once created, elements will be shown in infrastructure tab.

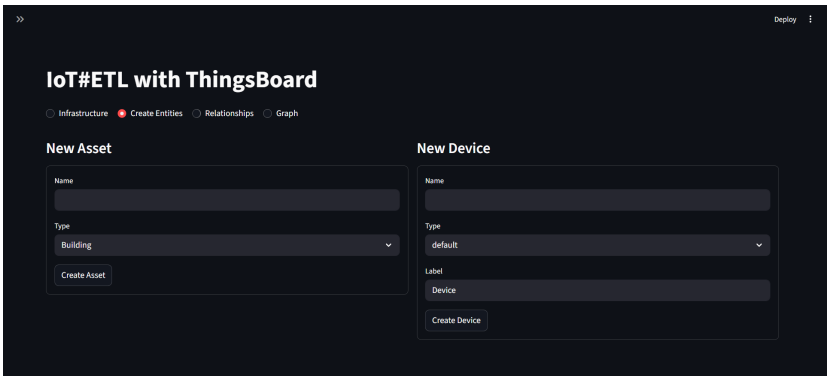

Figure 4: Infrastructure Tab

3. **Relationships** tab: its job is to show already synced relationships (from Thingsboard) and to give the tools to create new draft links between items, to then sync to the cloud.

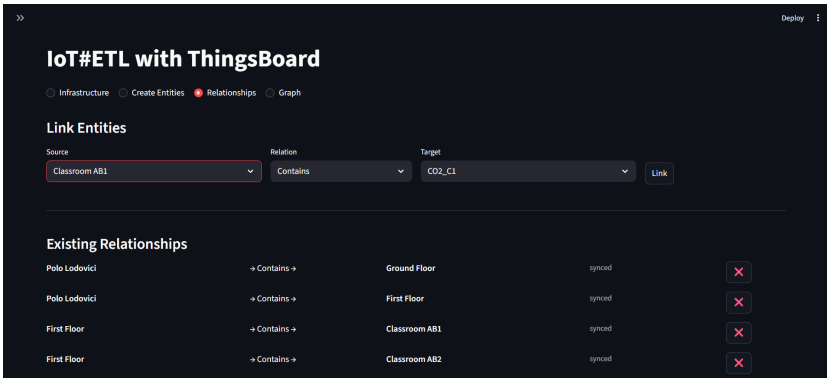

Figure 5: Relationships Tab

4. **Graph** tab: this just shows easily readable graph (just as Neo4j), giving more information about draft objects.

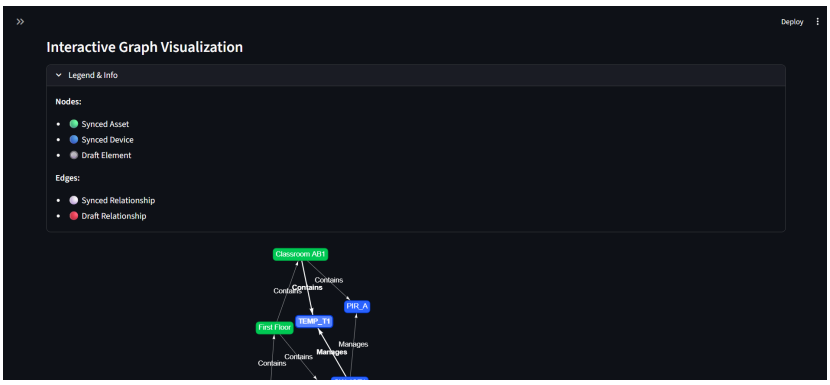

Figure 6: Graph Tab

System Testing Report

Date: `23/12/2025`
Tester: `Alessio Re`
Environment: `Final Testing for IoT#ETL with Thingsboard`

Table 1: Project Final Test Report

Test ID	Category	Scenario	Pre-Conditions	Expected Result	Status
T-001	Connectivity	Verify Service Connection	Docker containers (ThingsBoard, Neo4j) are up. .env is configured.	The Streamlit app launches without error. The sidebar loads without "Auth Failure" messages.	**PASS**
T-002	ETL (Import)	Import Cloud Data	ThingsBoard contains sample Assets, Devices, and Relations. Neo4j is empty.	Clicking "Import Cloud Data" populates Neo4j with all entities. Toast message confirms count (e.g., "✓5 Assets ✓3 Devices").	**PASS**
T-003	ETL (Import)	Import Existing Relations	Relationships exist in ThingsBoard.	Relationships appear in Neo4j with status synced. Push buttons are hidden in the UI.	**PASS**
T-004	Creation	Create Draft Asset	"Create Entities" tab selected.	New Asset appears in "Infrastructure" tab marked as draft. Appears as Grey node in Graph View.	**PASS**
T-005	Creation	Create Draft Device	"Create Entities" tab selected.	New Device appears in "Infrastructure" tab marked as draft. Appears as Grey node in Graph View.	**PASS**
T-006	Creation	Loop Detection	Two nodes exist. User tries to link Node A to Node A.	System prevents creation and displays "Loop detected" error.	**PASS**
T-007	Sync (Up)	Sync Draft Asset to Cloud	A draft Asset exists in Neo4j.	Clicking "Up Assets" pushes the entity to ThingsBoard. Neo4j status updates to synced (Node turns Green).	**PASS**

Test ID	Category	Scenario	Pre-Conditions	Expected Result	Status
T-008	Sync (Up)	Sync Draft Device to Cloud	A draft Device exists in Neo4j.	Clicking "Up Devices" pushes the entity to ThingsBoard. Neo4j status updates to synced (Node turns Blue).	**PASS**
T-009	Sync (Up)	Sync Relationship (Success)	Two synced nodes are linked by a draft relationship in Neo4j.	Clicking "Cloud Push" on the relationship creates the link in ThingsBoard. Edge status updates to synced (Line turns White).	**PASS**
T-010	Validation	Sync Relationship (Fail)	User tries to push a relationship where one node is still draft.	System blocks the request. Error message appears: "! Cannot sync... One or both entities are still Drafts."	**PASS**
T-011	Deletion	Safe Mode Deletion	"Deletion Policy" set to Safe Mode.	Deleting a node removes it from Neo4j only. The entity remains visible in the ThingsBoard dashboard.	**PASS**
T-012	Deletion	Strict Mode Deletion	"Deletion Policy" set to Strict Mode.	Deleting a node triggers a warning. Confirming "Yes" removes it from both Neo4j and ThingsBoard Cloud.	**PASS**
T-013	UI/UX	Visual Graph Rendering	Neo4j contains a mix of synced/-draft nodes and edges.	"Visual Graph" tab renders interactive topology. Legend matches colors (Green/Blue/Grey nodes, White/Red edges).	**PASS**
T-014	UI/UX	Delete Confirmation	User clicks "X" on any entity.	A confirmation dialog/popup appears asking "Are you sure?". Action is taken only after clicking "Yes".	**PASS**

Version 2

As a latter update of the system, there has been an implementation about fetching, storing and editing devices and assets attributes. In Thingsboard, there are three types of attributes: server-side, shared and client-side (https://thingsboard.io/docs/user-guide/attributes). Client-side attributes are directly related to the device and should only be available to it, so it's not showing in the system. These attributes are easily fetchable and are treated as key-value pairs. The new *Infrastructure* page (Figure 7) in the UI presents added options to manage attributes, furthermore these have been implemented in the *Graph* page, too (Figure 8).

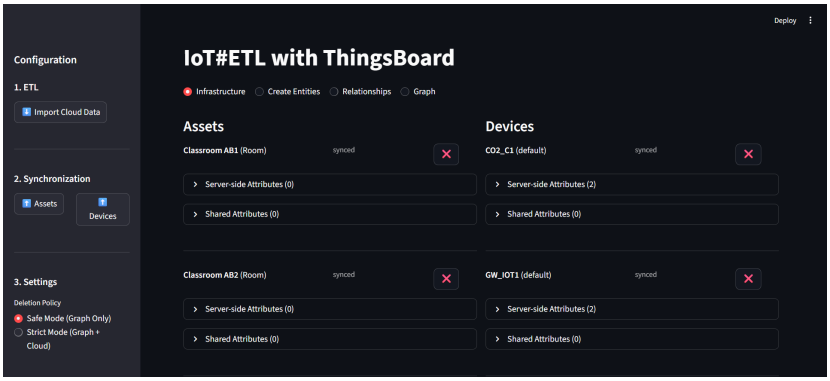

Figure 7: Infrastructure Tab (v2)

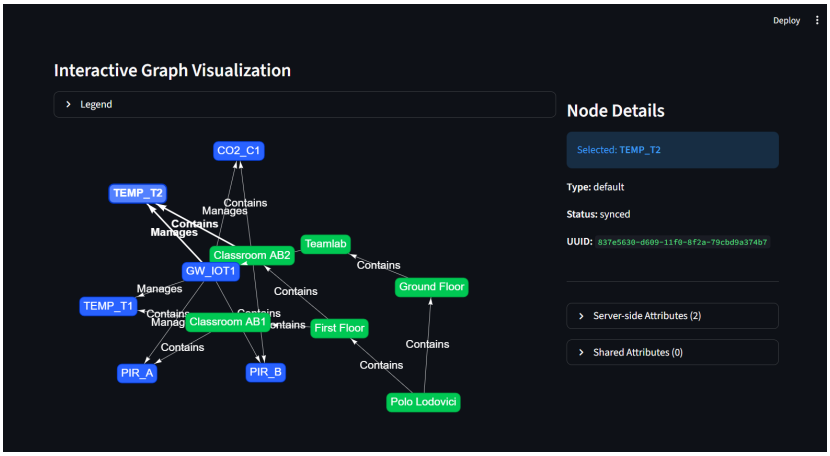

Figure 8: Graph Tab (v2)

Future Works and Maintainance

While the current implementation of *IoT#ETL with Thingsboard* successfully establishes a bidirectional bridge for topology management, several avenues for expansion and optimization remain. Future development will focus on transitioning from a static topology manager to a dynamic, real-time Digital Twin.

Planned Extensions

- **Real-Time Telemetry Visualization:** Currently, the graph visualizes static relationships. A key future enhancement is to overlay real-time telemetry data onto the graph nodes. By integrating ThingsBoard Websockets, the Streamlit dashboard could dynamically color-code nodes based on sensor thresholds and misurations.
- **Automated Synchronization (Webhooks):** The current ETL process relies on manual triggering via the UI. Future iterations could implement an event-driven architecture using ThingsBoard Rule Engine. By configuring Webhooks in ThingsBoard, creating or moving a device in the cloud could instantly trigger an update in Neo4j, eliminating the need for manual batch synchronization.
- **Conflict Resolution Mechanism:** As the system scales to multiple users, a concurrency control mechanism must be introduced to handle scenarios where the topology is modified simultaneously in ThingsBoard and the Graph Interface. A "Timestamp-based" resolution strategy or a locking mechanism will be necessary.

Maintenance & Scalability

To ensure the long-term reliability of the system, the following maintenance protocols are defined:

1. **API Versioning Compliance:** ThingsBoard evolves its REST API periodically. The ETL Middleware must be maintained to support new authentication flows and API structure changes to prevent service disruption (better intended as Regular Maintainance).
2. **Database Integrity Strategies:** Regular snapshots of the Neo4j database must be scheduled. Since the Graph acts as a persistence layer for the Digital Twin, implementing automated consistency checks will help detect "Ghost Nodes" that exist in the graph but were deleted directly from the cloud.
3. **Container Orchestration:** Currently deployed via `docker-compose`, the infrastructure should eventually be migrated to **Kubernetes** for production use. This would allow the Middleware to scale horizontally, handling thousands of asset updates per second without bottlenecking the synchronization process.

References

1. Streamlit. Available at: https://streamlit.io/
2. Thingsboard. Available at: https://thingsboard.io

3. Neo4j. Available at: https://neo4j.com/docs/operations-manual/current/docker/